

SIH26

Frontend-Backend End-to-End Verification Playbook

GitHub Copilot Agent test plan + repair loop

Production backend

Backend: <https://dropout-predictor-2pme.onrender.com>

API base: <https://dropout-predictor-2pme.onrender.com/api/v1>

Swagger: <https://dropout-predictor-2pme.onrender.com/docs>

Purpose: verify every important API contract from FastAPI -> Axios -> React UI, diagnose failures, fix the smallest root cause, and retest until the expected behavior is reached.

Source contract: SIH26 Frontend Master Prompt + Backend API Contract. If this playbook differs from the running FastAPI OpenAPI/Swagger contract, the running backend wins.

1. Test Objective and Definition of Done

The goal is not only to confirm that an HTTP endpoint returns 200. The goal is to confirm the complete chain: user action -> frontend API module -> Authorization header -> deployed FastAPI -> database/AI service -> response parsing -> correct React state -> correct UI rendering.

Do not create mock data just to make a page look successful. Real empty states are valid when the database legitimately has no records. A failure is when the frontend miscalls an endpoint, misreads the schema, fails to render valid data, exposes incorrect controls, or mishandles expected HTTP states.

PASS means all of these are true

1) Request uses the production API base. 2) Correct JWT is attached when required. 3) HTTP status is expected. 4) Response shape matches the live backend. 5) React renders the correct data/state. 6) Refresh preserves valid state. 7) Browser console has no relevant errors. 8) No fake business logic is duplicated in React.

Rule	Required behavior
Source of truth	Running FastAPI /docs or /openapi.json wins over older PDFs, mocks, or frontend assumptions.
API base	Use VITE_API_URL=https://dropout-predictor-2pme.onrender.com/api/v1. Do not scatter literal URLs through components.
Auth	JWT from /auth/login. Shared Axios interceptor attaches Authorization: Bearer <token>.
401	Clear auth state/token and redirect to /login.
403	Show unauthorized state. Do not loop/retry forever.
422	Show validation feedback using backend detail; do not silently change payload shape.
503	Show AI service temporarily unavailable; preserve user input where practical.
Student identity	Prefer /student-dashboard/me. Never let a student change student_code to inspect another student.
Tutor intelligence	Do not calculate correctness, BKT mastery, next question, diagnosis, or prerequisite routing in React.
EWS intelligence	Do not calculate risk tier or SHAP contribution in React.
Charts	Use actual API responses. Empty data -> intentional empty state, not fabricated bars/lines.

2. Copilot Verification Loop

1. Discover

Inspect frontend routes, src/api modules, auth context, environment variables, and the live OpenAPI contract.

2. Execute

Run the frontend and call the real deployed backend. Capture network status, response JSON, console errors, and rendered UI.

3. Compare

Compare actual response/behavior against the expected contract in this playbook and the live OpenAPI schema.

4. Classify

Decide whether the failure is URL/config, auth/RBAC, payload/schema, empty data, CORS, frontend mapping/state, AI service, or backend error.

5. Fix

Apply the smallest root-cause fix. Do not hardcode success, bypass auth, or replace the real backend with mocks.

6. Retest

Rerun the failed endpoint and its complete user flow. Also test refresh and one nearby regression path.

7. Record

Write PASS / FAIL / BLOCKED_EXTERNAL plus files changed and evidence into docs/e2e-verification-report.md.

Do not confuse empty data with broken integration

Example: /student-dashboard/me can validly return zero sessions or empty mastery history for a new student. The UI should show a polished empty state. That is PASS if the response is valid and the page behaves correctly.

3. Authentication and RBAC

ID	Method + path	Role	Expected API result	Expected frontend behavior
AUTH-01	POST /auth/login	Public	200; access_token, token_type, expires_in, user{user_id,email,display_name,role}	Token stored once; user state set; redirect by backend role. Invalid credentials -> clear error.
AUTH-02	GET /auth/me	Any authenticated	200; user_id,email,display_name,role,student_code,is_active	Used on refresh/startup. Page should restore session without forcing login again.
AUTH-03	GET /auth/users	ADMIN	200; user list	Admin table renders users, roles, status. Student/faculty should get 403 and no admin UI access.
AUTH-04	POST /auth/users	ADMIN	Created user; student_code required only for STUDENT	Create-user modal sends exact payload. Duplicate/invalid input surfaces backend validation.
AUTH-05	PATCH /auth/users/{user_id}/status	ADMIN	200; updated is_active	Status toggle updates row after successful response; rollback UI on failure.

4. Student Dashboard

ID	Method + path	Role	Expected API result	Expected frontend behavior
STU-01	GET /student-dashboard/me	STUDENT	student, support, learning_summary, recommended_next, recent_sessions, recent_tutor_activity	Student home shows support banner + KPI cards + recommendations + recent activity.
STU-02	GET /student-dashboard/me/mastery	STUDENT	student_code, threshold, concepts[] with mastery_probability/p	Mastery cards/progress bars use backend values only. Empty concepts

			percentage	-> empty state.
STU-03	GET /student-dashboard/me/mastery-trend?concept_id={id}	STUDENT	points[] with mastery_before/after, diagnosis, created_at	Line chart renders actual attempt sequence. 0 points -> no fabricated trend.
STU-04	GET /student-dashboard/me/recent-activity?limit=20	STUDENT	count + activity[]	Activity list renders diagnosis, question/concept, mastery change, local date formatting.
STU-05	GET /student-dashboard/{student_code}	FACULTY/ADMIN	Inspectable student dashboard	Faculty detail can inspect; STUDENT must not use this to access another student.
STU-06	GET /student-dashboard/{student_code}/mastery	FACULTY/ADMIN	Concept mastery for requested student	Faculty view renders selected student mastery.
STU-07	GET /student-dashboard/{student_code}/mastery-trend	FACULTY/ADMIN	Trend points for selected student/concept	Faculty chart renders or clean empty state.
STU-08	GET /student-dashboard/{student_code}/recent-activity	FACULTY/ADMIN	Recent student attempts	Faculty recent activity renders without impersonating student actions.

5. Adaptive Tutor

ID	Method + path	Role	Expected API result	Expected frontend behavior
TUT-01	GET /tutor/concepts	Available/current backend auth rules	count + concepts[]	Concept picker loads graph nodes and prerequisite indicators.
TUT-02	GET /tutor/mastery/{student_code}	Student own; faculty/admin inspect	Per-concept mastery state	Student own code is taken from auth state, not editable input.
TUT-03	GET /tutor/history/{student_code}?limit=50	Student own; faculty/admin inspect	count + interactions[]	Tutor history timeline renders diagnosis, response, mastery before/after.
TUT-04	GET /tutor/history/{student_code}/{concept_id}?limit=50	Student own; faculty/admin inspect	Concept-specific interactions	Concept filter returns only selected concept history.
TUT-05	GET /tutor/next-question/{student_code}/{concept_id}	Student learning flow	current_mastery + question{question_id, text, difficulty}	Question card displays exactly returned question; expected_answer never appears.
TUT-06	POST /tutor/adaptive-attempt	STUDENT own	evaluation + tutor + optional recommended_concept + mastery	Frontend sends student_code, question_id, student_answer, language_code only. Never sends

				is_correct/expected_answer.
TUT-07	POST /tutor/attempt	Legacy/test only	Backend-specific legacy response	Final UI should not depend on this route unless current backend explicitly requires it.

6. Learning Session Engine

Critical session rule: always render response.next. A prerequisite gap can change the active concept. Do not keep showing a stale question from local state.

ID	Method + path	Role	Expected API result	Expected frontend behavior
SES-01	POST /tutor/sessions/start	STUDENT owner	session_id,status,concept,target_questions,starting_mastery,next	Start page transitions to active session and renders response.next.question.
SES-02	GET /tutor/sessions/student/{student_code}?limit=20	Student own; faculty/admin inspect	Session history	Session history list persists after refresh.
SES-03	GET /tutor/sessions/{session_id}	Owner; faculty/admin inspect	Session summary with progress, mastery, diagnoses, attempts	Session details render current/completed state correctly.
SES-04	GET /tutor/sessions/{session_id}/next-question	Owner	Current/next backend-selected question	UI never computes local next question.
SES-05	POST /tutor/sessions/{session_id}/answer	Owner	attempt + progress + next	After submit: feedback, progress, mastery change, prerequisite redirect, next question.
SES-06	POST /tutor/sessions/{session_id}/end	Owner	Completed session summary	Manual end shows final summary and persists completion status.

7. Faculty Learning Analytics

ID	Method + path	Role	Expected API result	Expected frontend behavior
FAC-01	GET /faculty/learning/overview	FACULTY/ADMIN	students, risk, learning, faculty KPI groups	Overview KPI cards render totals, risk counts, average mastery, sessions, Socratic interactions, pending interventions.
FAC-02	GET /faculty/learning/weak-concepts?limit=10	FACULTY/ADMIN	concepts[] with tracked/below-mastery/average fields	Horizontal bar/ranking uses backend data; empty -> no weak-concept data state.
FAC-03	GET /faculty/learning/misconceptions?limit=20	FACULTY/ADMIN	misconceptions[]	Bar/ranked list renders diagnosis counts without inventing

				categories.
FAC-04	GET /faculty/learning/at-risk-students?limit=50	FACULTY/ADMIN	students[] with risk + average_mastery	At-risk table searchable/filterable; clicking row opens correct student.
FAC-05	GET /faculty/learning/student/{student_code}	FACULTY/ADMIN	student + risk + mastery + recent_sessions + latest_intervention	Combined student detail view renders EWS and learning context together.

8. EWS and Intervention Dashboard

Intervention approve/reject action paths must be discovered from the running /docs or /openapi.json before wiring or testing. Do not invent the route. Verify status changes PENDING_REVIEW -> APPROVED or REJECTED and ensure risk prediction itself is not rewritten by that decision.

ID	Method + path	Role	Expected API result	Expected frontend behavior
EWS-01	GET /dashboard/summary	FACULTY/ADMIN	total/evaluated + low/moderate/critical + intervention counts	Institution cards and risk distribution chart match counts exactly.
EWS-02	GET /dashboard/at-risk?limit=20	FACULTY/ADMIN	latest MODERATE/CRITICAL students	At-risk table shows risk tier, percentage, top factor, evaluated time.
EWS-03	GET /dashboard/student/{student_code}	FACULTY/ADMIN	student + latest_snapshot + risk + intervention	Detailed EWS view maps current GPA, attendance, assignment completion, risk, SHAP factors.
EWS-04	GET /dashboard/interventions? status=PENDING_REVIEW	FACULTY/ADMIN	Intervention queue rows	Queue shows route, plan, outreach, status, notes; filters do not mutate backend data.
EWS-05	GET /dashboard/student/{student_code}/risk-history	FACULTY/ADMIN	risk history[]	Line chart uses returned week/evaluated sequence and percentages.
EWS-06	POST /risk/evaluate-all?limit={n}&force=false	Prefer ADMIN	requested,evaluated,skipped,failed,results,failures	Never auto-run on page render. Explicit admin action only; show progress/result summary.

9. Parent WhatsApp Report - Conditional Feature

Only test this section if these routes exist in the live backend. Parent report generation/sending should remain faculty-triggered and should not automatically send based only on a risk score.

ID	Method + path	Role	Expected API result	Expected frontend behavior
PAR-01	GET /faculty/students/{student_code}/guardian	FACULTY/ADMIN	Guardian contact or expected 404 when missing	Student detail shows guardian state without crashing.

PAR-02	PUT /faculty/students/{student_code}/guardian	FACULTY/ADMIN	Saved guardian contact	Form normalizes country-code number and shows consent/opt-in clearly.
PAR-03	GET /faculty/students/{student_code}/parent-report/preview	FACULTY/ADMIN	application/pdf for eligible student	Preview opens valid PDF containing student/support data and charts; no secrets in URL.
PAR-04	POST /faculty/students/{student_code}/parent-report/send	FACULTY/ADMIN	Delivery record/status + WhatsApp message id when successful	Send is explicit and confirmed; guardian opt-in required; failures show useful error and do not claim SENT.

10. High-Value Response Assertions

Login: access_token is non-empty; user.role is one of STUDENT/FACULTY/ADMIN; frontend stores token under one consistent key and does not store demo passwords.

Student dashboard: student.student_code exists; support.level is one of ON_TRACK/EXTRA_SUPPORT/HIGH_SUPPORT/NOT_EVALUATED; learning_summary numeric fields render even when zero.

Tutor attempt: evaluation.is_correct is boolean; diagnosis is recognized enum; tutor.message exists; mastery.before/after are 0..1; recommended_concept can be null.

Session answer: progress.answered increments; progress.target is stable; response.next controls whether session is complete and what question/concept is next.

Faculty overview: risk.low + moderate + critical should be logically consistent with evaluated_students when backend semantics are latest-per-student.

EWS detail: risk_tier is LOW/MODERATE/CRITICAL; SHAP factors are displayed as predictive drivers, not causal reasons.

Risk batch: force=false by default; do not interpret skipped as failure when snapshot was already evaluated.

11. Failure Diagnosis and Repair Matrix

Observed failure	Likely cause	Required investigation / repair
No network request	Button/handler not wired, early return, route not mounted	Inspect event handler, React state, router. Confirm API module function is actually called.
404	Wrong path, wrong prefix, outdated contract	Compare request against live OpenAPI. Fix frontend path; do not invent backend route.
401	Missing/expired token, wrong storage key, interceptor not used	Inspect localStorage and request Authorization header. Ensure all protected calls use shared Axios client.
403	Wrong role or ownership	Verify /auth/me role/student_code. Fix frontend access control/UX; do not bypass backend RBAC.
422	Payload/schema mismatch	Inspect backend validation detail and current OpenAPI request schema. Fix

		field names/types/nullability.
500	Backend bug or bad production data	Capture response/Render logs if available. Do not hide with frontend fallback; report exact failing call.
503	Gemini/AI temporarily unavailable	Show retryable AI unavailable state; preserve answer/session context where possible.
200 but blank UI	Wrong response mapping, stale mock types, nested field mismatch	console.log actual response once, compare types/selectors, fix component mapping and null handling.
200 and [] / zero data	May be legitimate empty database state	Render intentional empty state. Only seed/import data if the demo requires it; do not fabricate frontend data.
CORS error	Backend allowed origins missing frontend origin	Verify browser error and backend CORS configuration; add exact dev/production frontend origins.
Chart crashes	Undefined/null array or wrong numeric key	Guard empty/null data, map exact backend fields, render empty-state card instead of fake points.
Render cold start timeout	Hosted service sleeping/cold start	Retry after initial wake-up; distinguish infrastructure delay from contract failure.

12. Mandatory End-to-End User Journeys

Student login -> dashboard

1. POST /auth/login with student credentials.
2. GET /auth/me returns STUDENT and linked student_code.
3. Redirect to /student.
4. GET /student-dashboard/me renders name, support state, learning KPIs.
5. Refresh browser; session restores without re-login.

Student learning session

6. Load /tutor/concepts.
7. Start session for a concept.
8. Render backend next question.
9. Submit natural-language answer.
10. Render evaluation + Socratic feedback + mastery change.
11. Follow backend prerequisite/next-question state.
12. Complete/end session and verify session history + mastery page update.

Faculty EWS

13. Login as FACULTY.
14. Load faculty overview + dashboard summary.
15. Open at-risk list and one student detail.
16. Verify risk tier, predictive factors, mastery, recent sessions and intervention.
17. Open risk history and intervention queue.
18. If approve/reject routes exist, change one test intervention and verify persisted status after refresh.

Admin

19. Login as ADMIN.

20. List users.
21. Create a non-production test user only if safe.
22. Deactivate/reactivate test user.
23. Verify student/faculty cannot access admin routes.

13. Recommended Automated Test Structure

Copilot should first reuse the project's existing test tooling. If there is no browser E2E framework, Playwright is a good fit because one suite can test both direct API requests and browser flows. Do not add a second competing framework if the project already uses Cypress/Playwright/Vitest effectively.

```
frontend/
  tests/
    e2e/
      auth.spec.ts
      student-dashboard.spec.ts
      tutor-session.spec.ts
      faculty-ews.spec.ts
      admin.spec.ts
      parent-report.spec.ts      # only if live routes exist
    helpers/
      auth.ts
      api.ts
  docs/
    e2e-verification-report.md
  .github/
    copilot-instructions.md
  prompts/
    sih26-e2e-verify.prompt.md
```

14. Test Environment Variables

VITE_API_URL=https://dropout-predictor-2pme.onrender.com/api/v1

```
# E2E-only secrets: keep in .env.e2e.local or CI secrets; never commit real passwords.
E2E_STUDENT_EMAIL=student@sih.demo
E2E_STUDENT_PASSWORD=<local secret>
E2E_FACULTY_EMAIL=faculty@sih.demo
E2E_FACULTY_PASSWORD=<local secret>
E2E_ADMIN_EMAIL=admin@sih.demo
E2E_ADMIN_PASSWORD=<local secret>
```

Security

Never write passwords, JWTs, WhatsApp tokens, Gemini keys, or database URLs into tests, screenshots, committed reports, or this prompt file. Read secrets from environment variables only.

15. GitHub Copilot Agent Master Prompt

Use this prompt in VS Code Copilot Chat in Agent mode. A ready-made .prompt.md file is supplied with this PDF.

Act as the integration-test and repair agent for the SIH26 AI Student Success frontend.

Production backend:

<https://dropout-predictor-2pme.onrender.com>

API base:

<https://dropout-predictor-2pme.onrender.com/api/v1>

Swagger:

<https://dropout-predictor-2pme.onrender.com/docs>

MISSION

Verify every frontend-to-backend integration described in docs/SIH26_Frontend_Backend_E2E_Verification_Playbook.pdf and in the live FastAPI OpenAPI contract. For each endpoint, confirm both the HTTP/API behavior and the user-visible React behavior. If a test fails, diagnose the root cause, fix the smallest correct part of the code, rerun the failed test and related regression test, and continue until PASS or a genuine external blocker is identified.

NON-NEGOTIABLE RULES

- The running FastAPI OpenAPI contract is the source of truth. Do not invent endpoint paths or payload fields.
- Use VITE_API_URL and the shared Axios client. Do not scatter hardcoded backend URLs.
- Do not replace real API integration with mocks to make tests pass.
- Do not duplicate XGBoost risk logic, SHAP, BKT, Gemini evaluation, next-question logic, prerequisite routing, or backend RBAC in React.
- Never send expected_answer or is_correct from the student Tutor UI.
- Treat valid empty arrays/zero counts as an empty-state UX case, not automatically as a backend failure.
- Never expose or commit passwords, JWTs, database URLs, Gemini keys, or WhatsApp tokens.
- For intervention approve/reject and Parent Report routes, discover the exact live endpoint before testing. If a route does not exist, mark it NOT_IMPLEMENTED instead of inventing it.

WORKFLOW

1. Inspect package.json, routes, auth context/store, src/api, environment usage, and existing tests.
2. Retrieve/inspect the live OpenAPI schema and build an actual endpoint inventory.
3. Compare frontend API functions with the live paths, methods, request schemas, response shapes, and RBAC.
4. Reuse the existing test framework. If no browser E2E framework exists, add Playwright with minimal configuration.
5. Create direct API smoke tests and browser E2E tests for Auth, Student Dashboard, Mastery, Tutor, Learning Sessions, Faculty Analytics, EWS, Interventions, Admin, and Parent Reports if implemented.
6. Use credentials only from E2E_* environment variables.
7. Run tests against the deployed backend and local frontend.
8. For every failure, capture status code, response body, console/network error, expected behavior, actual behavior, and suspected root cause.
9. Fix only the root cause. Retest until PASS.
10. Create docs/e2e-verification-report.md containing a table: Test ID | Endpoint | Role | API result | UI result | Status | Fix applied | Files changed | Notes.
11. At the end, run frontend build/lint/tests and summarize remaining BLOCKED_EXTERNAL or NOT_IMPLEMENTED items.

ACCEPTANCE

- Authentication and role redirects work.
- Protected requests carry Bearer JWT.
- 401/403/404/422/503 states behave correctly.
- Student /me pages render real backend data and safe empty states.
- Tutor session persists answers, feedback, mastery, next-question behavior and history.
- Faculty analytics/EWS charts/tables use real API data.
- Students cannot access another student's data.
- Faculty cannot impersonate student Tutor answers.

- Admin-only user management is protected.
- No relevant browser console errors remain.
- Production build succeeds.
- Verification report is complete.

Do not stop after only analyzing. Execute the tests, make safe code fixes when needed, rerun, and update the report after each repair cycle. Ask me before any destructive database operation or production data deletion.

16. How to Give This to GitHub Copilot in VS Code

24. Open the frontend repository root in VS Code. If backend code is in another repository and you want Copilot to inspect it too, open both folders in one VS Code workspace; otherwise the live OpenAPI contract is enough for frontend integration testing.
25. Create `.github/copilot-instructions.md` for short, always-on project rules. VS Code automatically applies repository-wide Copilot instructions from this file.
26. Create `.github/prompts/sih26-e2e-verify.prompt.md` and paste the reusable master prompt supplied with this package.
27. Place this PDF under `docs/SIH26_Frontend_Backend_E2E_Verification_Playbook.pdf` so the test plan stays with the repository. For best AI readability, also keep the prompt as Markdown rather than relying only on PDF parsing.
28. Open Copilot Chat and select Agent mode (local agent). Invoke the prompt by typing `/sih26-e2e-verify`, or use Attach Context -> Prompt and choose the prompt file. If your VS Code version requires it, enable prompt files in settings.
29. Allow terminal commands and file edits when Copilot requests permission. Keep destructive actions manual. Do not give Copilot production secrets in chat; set `E2E_*` variables locally.
30. Review `docs/e2e-verification-report.md` after each run. Re-run `/sih26-e2e-verify` after major backend/frontend changes to perform a regression pass.

Recommended first Copilot message

Run `/sih26-e2e-verify`. Start with discovery only for 2-3 minutes: show me the live endpoint inventory, current frontend API modules, missing/mismatched routes, and the test execution plan. Then proceed to execute and repair the integration in small batches: Auth -> Student -> Tutor/Sessions -> Faculty/EWS -> Admin -> Parent Report.

17. Final Acceptance Checklist

- ☐ Production `VITE_API_URL` is correct and used by one shared client.
- ☐ Student, faculty and admin logins work with correct role redirects.
- ☐ `/auth/me` restores session on refresh.
- ☐ Student dashboard, mastery, trend and activity pages handle both populated and empty data.
- ☐ Tutor concept selection, question loading, adaptive answer, Socratic feedback and mastery update work.
- ☐ Learning session start/answer/next/end/history flow works without locally inventing questions.
- ☐ Faculty overview, weak concepts, misconceptions, at-risk list and student detail render real data.
- ☐ EWS summary, at-risk list, detail, risk history and interventions render real data.
- ☐ Intervention status updates persist using the actual live approve/reject routes.
- ☐ Admin user list/create/status controls are backend-protected.
- ☐ Parent report guardian/preview/send works only if implemented; PDF is valid and send state is audited.
- ☐ Charts use API data and have clean loading/empty/error states.
- ☐ No hardcoded student IDs when `/me` is available.

- [] No expected answers, correctness, risk logic, SHAP logic or BKT logic duplicated in React.
- [] No secrets are committed.
- [] npm build/lint/test and E2E suite pass, or remaining external blockers are explicitly documented.

End of verification playbook